

TRABAJO PRÁCTICO N.º 1

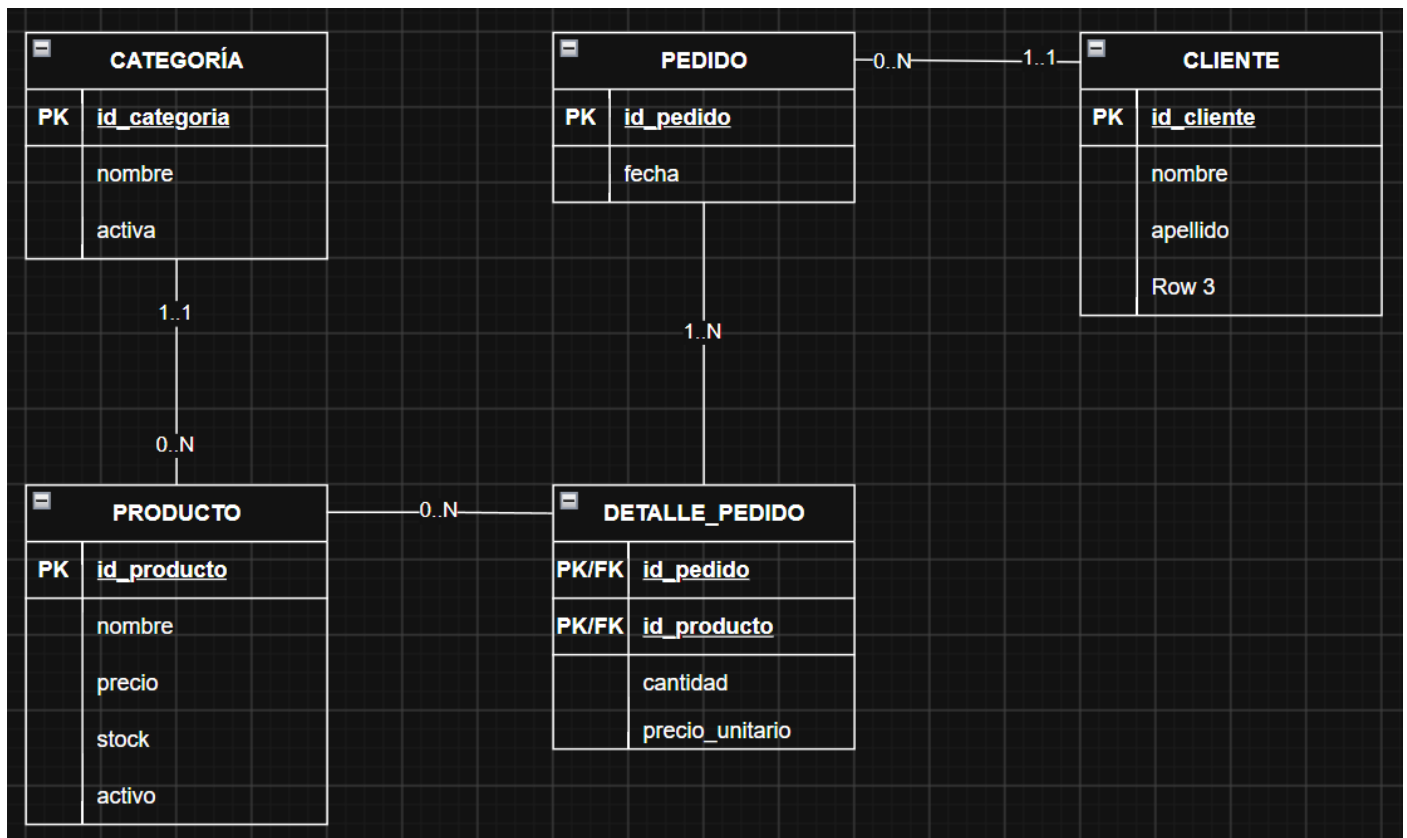
Del dominio al esquema: modelo entidad-relación, modelo relacional, normalización y DDL

Parte 1 - Modelo entidad-relación

Diccionario de Entidades

Entidad	Atributo	Tipo conceptual	Clave / restricción	Descripción
categoria	id_categoria	Numérico	PK	Identificador único de la categoría
	nombre	Texto	Único	Nombre de la categoría, por ejemplo "Pizzas"
	activa	Booleano	—	Indica si la categoría está activa
producto	id_producto	Numérico	PK	Identificador único del producto
	nombre	Texto	—	Nombre del producto
	precio	Numérico	≥ 0	Precio actual de lista
	stock	Numérico	≥ 0	Cantidad disponible
	activo	Booleano	—	Indica si el producto está activo
cliente	id_cliente	Numérico	PK	Identificador único del cliente
	nombre	Texto	—	Nombre del cliente
	apellido	Texto	—	Apellido del cliente
	email	Texto	Clave candidata / UNIQUE	Correo electrónico único del cliente
pedido	id_pedido	Numérico	PK	Identificador único del pedido
	fecha	Fecha/fecha-hora	—	Fecha en que se realizó el pedido
detalle_pedido	id_pedido	Numérico	PK/FK	Pedido al que pertenece el detalle
	id_producto	Numérico	PK/FK	Producto incluido en el pedido
	cantidad	Numérico	> 0	Unidades solicitadas
	precio_unitario	Numérico	≥ 0	Precio del producto al momento del pedido

Diagrama ER



Preguntas Guía

a) ¿Por qué la relación entre producto y pedido no puede resolverse como una relación binaria simple 1:N? ¿Qué información se perdería si lo intentaras?

Porque la relación entre pedido y producto es naturalmente N:M. Un pedido puede contener muchos productos, y un mismo producto puede formar parte de muchos pedidos diferentes.

Además, la relación necesita almacenar información que depende de ese producto dentro de ese pedido en particular, principalmente:

- cantidad solicitada;
- precio unitario facturado.

Si intentáramos resolverla como una relación 1:N, no podríamos representar correctamente que un mismo producto aparece en muchos pedidos. También perderíamos la posibilidad de almacenar correctamente el precio histórico. Por ejemplo, si una pizza costaba \$5000 en marzo y \$6000 en abril, el pedido realizado en marzo debe conservar `precio_unitario = $5000`, aunque actualmente el producto tenga precio \$6000. Por eso se necesita `detalle_pedido` como entidad asociativa.

b) ¿Qué entidad tiene participación parcial en la relación con categoría, y cuál tiene participación total? ¿Qué pasaría si invirtieras esa lectura?

En la relación Categoría — Producto, la Categoría tiene participación parcial, porque puede existir una categoría que todavía no tenga ningún producto asociado, esto está establecido en R1. El

Producto tiene participación total, porque todo producto debe pertenecer exactamente a una categoría.

Si se invirtiera la lectura, se estaría afirmando que existen productos sin categoría y que toda categoría necesariamente tiene al menos un producto. Ambas afirmaciones contradicen la regla R1 y modificarían incorrectamente las reglas del negocio.

c) ¿Alguno de tus atributos podría descomponerse en partes más simples (por ejemplo, un nombre completo en nombre y apellido)? Justificá tu decisión de dejarlo compuesto o de separarlo.

Sí. El nombre completo del cliente podría almacenarse como un único atributo `nombre_completo`, pero decidí separarlo en:

- nombre
- apellido

Esto permite consultar, ordenar o filtrar independientemente por nombre o apellido y evita almacenar dos datos conceptualmente diferentes dentro de un mismo atributo.

El email, en cambio, se mantiene como un atributo simple porque, para este dominio, interesa como una única dirección de correo electrónico y además funciona como clave candidata, ya que R6 establece que debe ser único.

Parte 2 — Derivación al modelo relacional

Tablas Derivadas

categoría (id_categoria, nombre, activa)

producto (id_producto, nombre, precio, stock, activo, id_categoria -> categoría)

detalle_pedido (id_pedido -> pedido, id_producto -> producto, cantidad, precio_unitario)

pedido (id_pedido, fecha, id_cliente -> cliente)

cliente (id_cliente, nombre, apellido, email)

Preguntas Guía

a) ¿Qué pasaría con la integridad de tus datos si la tabla intermedia no incluyera ambas claves foráneas como NOT NULL?

Si `detalle_pedido` no tuviera ambas claves foráneas como NOT NULL, podrían existir registros incompletos.

Por ejemplo, podría aparecer:

id_pedido	id_producto	cantidad
100	NULL	2

En ese caso sabríamos que se solicitaron dos unidades, pero no sabríamos qué producto corresponde a esa línea. También podría ocurrir lo contrario, donde sabríamos qué producto fue solicitado, pero no a qué pedido pertenece.

Esto rompe la integridad del modelo porque una fila de detalle_pedido representa la asociación entre un pedido concreto y un producto concreto. Por lo tanto, ambas FK deben ser NOT NULL.

Además, al formar conjuntamente la PK (id_pedido, id_producto), tampoco pueden ser NULL desde el punto de vista de la definición de clave primaria.

b) Si el pedido pudiera existir sin ningún producto asociado (por ejemplo, un pedido recién iniciado), ¿cambia esto la participación que definiste en la Parte 1 para la relación con la tabla intermedia? Explica el impacto.

Sí, cambia la participación del pedido en la relación con detalle_Pedido. En la Parte 1 habíamos considerado que un pedido incluye productos, por lo que la participación del pedido podía interpretarse como total/obligatoria respecto de sus detalles.

Pero si ahora se permite crear un pedido recién iniciado que todavía no tiene ningún producto, entonces, un Pedido puede tener 0 o muchos detalle_Pedido. Por lo tanto, la participación de pedido pasa a ser parcial/opcional. La relación quedaría:

pedido (1..1) --- (0..N) detalle_Pedido

Esto no significa que detalle_pedido.id_pedido pueda ser NULL. Es justamente lo contrario: cada detalle existente debe pertenecer a un pedido. La diferencia está en si un pedido puede existir sin detalles, no en si un detalle puede existir sin pedido.

Parte 3 — Normalización hasta 3FN/BCNF

Clave Candidata:

- (nro_pedido, producto)

Dependencias Funcionales

- nro_pedido -> fecha, cliente, forma_pago
- producto -> categoría
- (nro_pedido, producto) -> cantidad, precio_unitario, subtotal

Verificación 1FN

Una relación está en Primera Forma Normal (1FN) cuando sus atributos contienen valores atómicos, es decir, no hay grupos repetitivos ni atributos con múltiples valores dentro de una misma celda. La relación ya se encuentra en 1FN. No es necesario realizar ninguna modificación en este paso.

Verificación 2FN

Para estar en 2FN, además de estar en 1FN, ningún atributo no clave puede depender solamente de una parte de una clave primaria compuesta, a esto se lo conoce como dependencia parcial.

Atributos como fecha, cliente, forma_pago y categoría dependen solo de una parte de la clave compuesta.

Separamos los atributos que dependen solamente de nro_pedido:

- pedido (nro_pedido, fecha, cliente, forma_pago)

Separamos el atributo que depende solamente de producto:

- producto (nombre_producto, categoría)

Quedandonos detalle_pedido de la siguiente forma:

- detalle_pedido (nro_pedido, producto, cantidad, precio_unitario, subtotal)

Verificación 3FN

Para estar en 3FN, además de estar en 2FN, ningún atributo no clave puede depender de otro atributo no clave, a esto se lo conoce como dependencia transitiva.

No hay atributos no clave que dependan de otros atributos no clave, por lo que las 3 tablas cumplen 3FN.

Verificación BCNF

Una relación está en BCNF si no posee redundancias basadas en dependencias funcionales.

Todas las tablas resultantes cumplen BCNF, porque en cada dependencia funcional relevante el determinante es una clave candidata de su respectiva tabla.

Conjunto final de tablas normalizadas

- pedido (nro_pedido, fecha, cliente, forma_pago)
- producto (nombre_producto, categoría)
- detalle_pedido (nro_pedido, producto, cantidad, precio_unitario, subtotal)

Preguntas de Integración

a) Comparé las tablas que obtuviste normalizando la planilla con las tablas que derivaste en la Parte 2 a partir de tu modelo ER. ¿A qué entidad de tu ER corresponde cada tabla? ¿Hay atributos de tu ER (por ejemplo, el mail del cliente) que no aparecían en esta planilla? ¿Qué te dice eso sobre la diferencia entre modelar a partir de un enunciado de negocio y normalizar a partir de datos históricos ya existentes?

Las tablas obtenidas mediante normalización coinciden conceptualmente con varias de las entidades que habíamos obtenido a partir del modelo ER:

Normalización de la Planilla	Modelo ER de Parte 2
PEDIDO	Pedido
PRODUCTO	Producto
DDETALLE_PEDIDO	Detalle_Pedido
cliente dentro de PEDIDO	Cliente
categoria dentro de PRODUCTO	Categoria

Sin embargo, hay una diferencia importante: en la planilla utilizada para normalizar no aparecen todos los atributos que habíamos definido en el modelo ER. Por ejemplo, en nuestro modelo ER, Cliente tenía: id_cliente, nombre, apellido y email. Mientras que en la planilla aparece solamente el nombre del cliente. Lo mismo ocurre con otros atributos que agregamos en el modelo ER, como activo, stock y el precio actual del producto.

Esto demuestra que modelar a partir de un enunciado de negocio y normalizar datos existentes son procesos relacionados, pero diferentes. El modelo ER intenta representar el dominio completo y sus reglas de negocio, incluso información que todavía no aparece en los datos históricos. La normalización, en cambio, parte de los atributos y dependencias que existen en una relación determinada y busca eliminar redundancias y anomalías.

b) El atributo subtotal de cada línea es, en teoría, calculable como $\text{cantidad} \times \text{precio_unitario}$. ¿Conviene almacenarlo igual, o siempre recalcularlo al vuelo? ¿Esa decisión de diseño viola alguna forma normal, o es un tema distinto (rendimiento, trazabilidad histórica)? Justificá en dos o tres líneas.

El atributo subtotal puede calcularse como: $\text{subtotal} = \text{cantidad} \times \text{precio_unitario}$

Por lo tanto, no es estrictamente necesario almacenarlo, ya que puede calcularse al consultar los datos. No almacenarlo evita redundancia y el riesgo de que subtotal quede inconsistente con cantidad o precio_unitario.

Sin embargo, almacenarlo podría justificarse por cuestiones de rendimiento o trazabilidad histórica en determinados diseños. La decisión de almacenarlo no constituye por sí misma una violación de 1FN, 2FN, 3FN o BCNF. Es principalmente una decisión de diseño relacionada con redundancia, rendimiento y consistencia.